Move Semantics

Value categories, what `std::move` really does, writing move operations, copy elision, and the traps. Assumes OOP (the Rule of Five).

Lvalues, rvalues and value categories

An **lvalue** has a name and an address (`x`, `v[0]`). An **rvalue** is an unnamed temporary (`5`, `a + b`, a returned value): about to vanish, so its contents can be stolen safely.

CATEGORY	ROUGHLY	EXAMPLES
lvalue	a named object	`x`, `v[0]`
prvalue	an unnamed temporary	`5`, `a + b`, a call that returns by value
xvalue	an object marked as movable	`std::move(x)`

"Rvalue" means prvalue or xvalue. Which reference a value can bind to:

REFERENCE TYPE	BINDS TO	TYPICAL USE
`T&`	lvalues only	modify the caller's object
`const T&`	lvalues **and** rvalues	read-only, avoid a copy
`T&&` (rvalue reference)	rvalues only	move constructors and move assignment

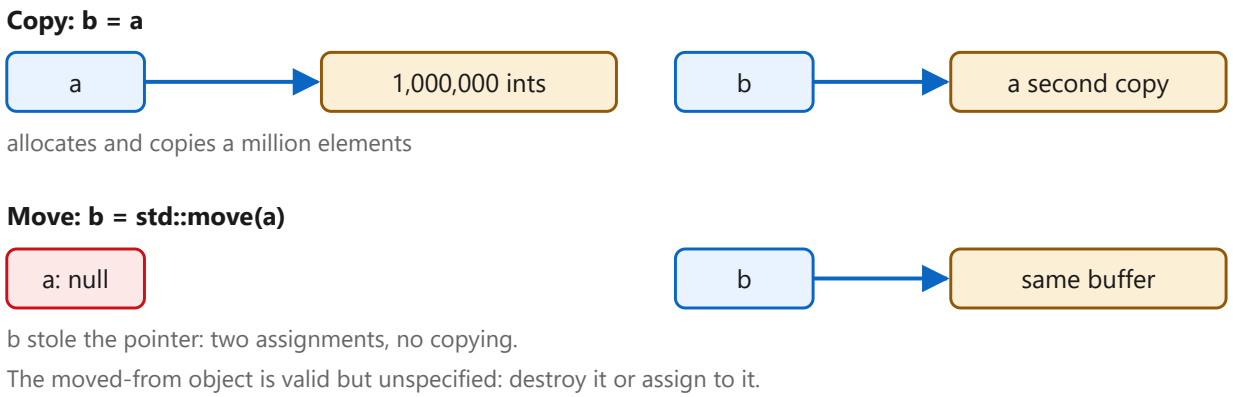
```cpp
void f(const std::string& s);   // accepts anything, copies nothing
// accepts only temporaries: "I may take your contents"
void g(std::string&& s);

std::string name = "abc";
f(name);              // OK
// g(name);           // ERROR: name is an lvalue
g(std::string("x"));  // OK: temporary
g(std::move(name));   // OK: std::move says "treat name as a temporary"

int x = 0;
// double parentheses report the category:
static_assert(std::is_same_v<decltype((x)),           int&>);
// lvalue int&, xvalue int&&, prvalue plain int
static_assert(std::is_same_v<decltype((std::move(x))), int&&>);
static_assert(std::is_same_v<decltype((x + 1)),       int>);
```

A NAME IS ALWAYS AN LVALUE Inside `g`, `s` is an lvalue despite its `&&` type. Pass it on with `std::move(s)` (or `std::forward`, see Templates).

Copy versus move



std::move only casts; the move constructor does the stealing.

Copying a million-element vector copies a million elements. If the source is a dying temporary, that is wasted work. A **move** just *steals the internal pointer*.

- `std::move(x)` **moves nothing**: it is a cast to an rvalue reference meaning "you may steal". The move constructor or assignment does the stealing.

- Overload resolution only looks at the *kind* of expression: an lvalue picks the copy, an rvalue picks the move.
- A moved-from object is **valid but unspecified**: destroy it or assign a new value, do not read it. Compiled: a moved-from `std::string` had size 0, and assigning to it worked.

> KEY POINT "What does `std::move` do?" Nothing at runtime: an unconditional cast to an rvalue reference. The move constructor does the work.

Writing move operations

A class that owns a raw resource needs the Rule of Five (OOP lesson). The move operations steal and leave the source safely empty:

```
class Buffer {
    int* data_; std::size_t n_;
public:
    // steal and reset, in one line each
    Buffer(Buffer&& o) noexcept
        : data_(std::exchange(o.data_, nullptr)), n_(std::exchange(o.n_, 0))
{}

    // copy-and-swap: by value covers copy AND move
    Buffer& operator=(Buffer o) noexcept {
        std::swap(data_, o.data_); std::swap(n_, o.n_);
        return *this;
    }
    /* ... constructor, destructor, copy constructor ... */
};
```

- `std::exchange(x, v)` returns the old `x` and sets it to `v`: the idiom for "steal and reset".
- **Mark moves** `noexcept`. `std::vector` moves its elements on reallocation only if the move cannot throw; otherwise it copies them (exception safety).
- The by-value `operator=` handles self-assignment and gives the strong guarantee. The classic pair `operator=(const T&)` and `operator=(T&&)` is faster when you can reuse the existing storage.

- If you can avoid owning a raw resource, do (Rule of Zero): `vector` and `unique_ptr` members make the compiler-generated moves correct.

Returning objects: elision and `std::move`

Compiled with a counting type (copies / moves performed):

WHAT YOU WRITE	COPIES	MOVES	WHY
`return Counter();`	0	0	guaranteed elision (C++17): built in the caller's storage
`Counter c; return c;`	0	0	NRVO: allowed, not required, but done in practice
`return std::move(c);`	0	**1**	blocks NRVO: an extra move, so never write it
return a by-value **parameter**	0	1	no elision for parameters, but it is moved, not copied
`return member;`	**1**	0	a member is never moved implicitly
`return std::move(member);`	0	1	here `std::move` is right: the object keeps living, you take its member

```
// fine: elided or moved automatically
Buffer make()           { Buffer b(1000); return b; }
// member of an object that outlives the call: move it out
Buffer take(Source& s) { return std::move(s.buffer); }
```

> RULE Return locals and temporaries plainly. Use `std::move` on a **member**, or on something that is not the local you are returning by name.

Sink parameters and move traps

Taking a parameter you will store

```
class Person {
    std::string name_;
public:
    // by value, then move into the member
    explicit Person(std::string n) : name_(std::move(n)) {}
};
```

Compiled with a counting type: `Holder(lvalue)` and `Holder(rvalue)` together made **1 copy and 2 moves**: the lvalue is copied once into the parameter, and each call moves once into the member. For a **constructor that stores its argument**, take it by value and move. Not for setters: the extra construction can cost more than assigning.

Traps

TRAP	WHAT HAPPENS
`std::move` on a `const` object	**copies**: a `const T&&` cannot bind to `T&&`, so the copy constructor is chosen (compiled: 1 copy, 0 moves)
a type with no move operations	silently copies
using an object after `std::move`	valid but unspecified; only destroy or assign to it
`std::move` on a forwarding reference parameter	moves an lvalue the caller still wanted: use `std::forward`
moving inside a loop	the second iteration sees a moved-from object

`push_back(lvalue)` copies, `push_back(temporary)` moves, `emplace_back(args ... )` builds in place (compiled together: 1 copy, 1 move).

MOVE ONLY THE LAST USE After `auto b = std::move(a);` treat `a` as empty. Never return `const` values by value: it disables the move.

Perfect forwarding

This builds on the forwarding references of the Templates lesson (`T&&` with a deduced `T` binds to anything and remembers whether it was an lvalue or an rvalue) and on `std::move` above.

A wrapper's parameter has a *name*, so it is an lvalue and the callee would always copy. `std::forward` restores the category:

```
template <typename T, typename ... Args>
// forwarding references
std::unique_ptr<T> makeUnique(Args&& ... args) {
    // lvalues stay lvalues, rvalues stay rvalues
    return std::unique_ptr<T>(new T(std::forward<Args>(args) ... ));
}

std::string s = "abc";
// s is an lvalue -> Widget copies it
auto a = makeUnique<Widget>(s);
// temporary rvalue -> Widget MOVES it
auto b = makeUnique<Widget>(std::string("x"));
```

STD::MOVE VS STD::FORWARD `move` *always* casts to rvalue. `forward<T>` casts to rvalue *only if* the argument was one. Use `forward` only on a forwarding reference, once.

Further reading by section

Where each section of this lesson is explained in more depth. Pages are PDF viewer pages (the number your PDF reader shows).

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Lvalues, rvalues and value categories	C++ Primer	4.1 Fundamentals	163 to 167
	C++ Primer	13.6 Moving objects	560 to 579
	C++ Move Semantics	8.1.2 to 8.1.3 and 8.4 to 8.5 Value categories since C++11 and C++17; when lvalues become rvalues and the reverse	145 to 148 and 153 to 154
	C++ Move Semantics	8.2 to 8.3 and 8.6 Special value-category rules; binding references; checking with decltype	148 to 153 and 154 to 156
Copy versus move	C++ Primer	13.6 Moving objects	560 to 579
	Effective Modern C++	Item 23, Understand std::move and std::forward	176 to 182
	C++ Move Semantics	1.1 Motivation for move semantics	21 to 36
	C++ Move Semantics	1.2 to 1.4 Implementing move semantics; copying as a fallback; const objects	36 to 41
	C++ Move Semantics	Chapter 2 Core features of move semantics	43 to 52
	C++ Move Semantics	6.1 to 6.2 Moved-from states (required and guaranteed states; destructible and assignable)	107 to 115
Writing move operations	C++ Primer	13.6 Moving objects	560 to 579
	Programming: Principles and Practice	17.4 to 17.5 Copying and moving; essential operations	501 to 512
	Programming: Principles and Practice	17.6 to 17.9 Other operations, remaining Vector problems, changing size	513 to 523
	Effective Modern C++	Item 14, Declare functions noexcept if they won't emit exceptions	108 to 115
	C++ Move Semantics	6.3 Dealing with broken invariants	115 to 124
Returning objects: elision and std::move	Effective Modern C++	Item 25, Use std::move on rvalue references, std::forward on universal references	186 to 195

SECTION	BOOK	PART OF THE BOOK	PDF PAGES
Sink parameters and move traps	Effective Modern C++	Item 29, Assume move operations are not present, not cheap, and not used	221 to 225
	Effective Modern C++	Item 41, Consider pass by value for copyable, cheap-to-move parameters that are always copied	299 to 310
	C++ Move Semantics	4.1 to 4.2 Avoid objects with names; avoid unnecessary std::move	77 to 79
	C++ Move Semantics	4.3.1 to 4.3.4 Initializing members with move semantics	79 to 90
	C++ Move Semantics	4.3.5 to 4.3.6 Member initialization summary; always pass by value and move?	90 to 93
Perfect forwarding	C++ Move Semantics	9.1 to 9.3 Perfect forwarding; universal versus rvalue references	159 to 166
	C++ Move Semantics	9.4 to 9.5 Overload resolution with universal references; perfect forwarding in lambdas	167 to 170
	C++ Move Semantics	Chapter 11 Perfect passing with auto&&	191 to 202